

Key-Value Store v1

Multithread, sezioni critiche, safety e liveness

Architetture dei Sistemi Distribuiti

Lezione 10

Che Cosa Cambia Rispetto Al v0

L'interfaccia verso il client cambia poco. L'architettura interna cambia molto:

- un thread per connessione;
- piu' client simultanei;
- stato condiviso nel processo;
- interleaving non controllati automaticamente.

Il problema nuovo non e' la forma dei comandi, ma la correttezza della loro esecuzione concorrente.

Perche' Questa Tappa E' Importante

La stessa API del v0 non conserva automaticamente la stessa semantica. In presenza di concorrenza diventano visibili:

- race condition;
- lost update;
- letture su stato transitorio;
- lock troppo larghi che degradano il progresso.

Introduciamo INCR <key> per rendere esplicito un percorso read-modify-write:

- 1 leggere il valore corrente;
- 2 interpretarlo;
- 3 produrre un nuovo valore;
- 4 riscriverlo nello stato condiviso.

Questo e' il primo vero esempio in cui una singola operazione applicativa corrisponde a piu' accessi allo stato.

Conviene classificare le operazioni:

- **read-only**: GET, EXISTS, KEYS;
- **write semplici**: SET, DELETE;
- **read-modify-write**: INCR.

La famiglia piu' delicata e' la terza, ma anche le altre richiedono una disciplina coerente rispetto allo stato condiviso.

Interleaving Unsafe Classico

Stato iniziale:

```
counter = 10
```

Interleaving possibile:

```
T1: read counter -> 10  
T2: read counter -> 10  
T1: write counter <- 11  
T2: write counter <- 11
```

Risultato: un incremento si perde.

Il problema non e' solo che il sistema "fa una cosa strana". E' una violazione di una proprieta' desiderata:

- due INCR completati dovrebbero produrre un incremento totale di 2;
- il valore finale 11 invece di 12 mostra una storia impossibile rispetto al contratto desiderato;
- il server resta vivo, ma si comporta in modo scorretto.

```
current = self._data.get(key, "0")
numeric_value = int(current)
time.sleep(INCR_DELAY_SECONDS)
numeric_value += 1
self._data[key] = str(numeric_value)
```

La vulnerabilit  non   sulla singola assegnazione finale.   sull'intera sequenza logica:

- leggo;
- assumo che il valore resti valido;
- scrivo in base a quell'assunzione.

Il punto tecnico importante e':

- non basta rendere atomica la singola write;
- va resa atomica l'intera transizione read-modify-write;
- la sezione critica deve coprire tutto cio' che dipende da una lettura condivisa.

Questa e' una lezione generale, non solo del KV store.

```
with self._lock:
    current = self._data.get(key, "0")
    numeric_value = int(current)
    time.sleep(INCR_DELAY_SECONDS)
    numeric_value += 1
    self._data[key] = str(numeric_value)
```

Il lock qui non protegge una struttura dati in astratto. Protegge una transizione logica del contratto del servizio.

Usiamo *safety* nel senso:

non accade mai qualcosa di scorretto

Esempi concreti:

- nessun lost update su INCR;
- nessuna osservazione di stato parziale;
- nessuna storia incompatibile con la disciplina di serializzazione scelta.

Usiamo *liveness* nel senso:

il sistema continua a poter fare progresso

Esempi concreti:

- i thread non si bloccano per sempre;
- nuove richieste continuano a essere servite;
- il lock non deve trasformarsi in un collo di bottiglia ingestibile.

Perche' Safety E Liveness Vanno Insieme

Una soluzione puo' migliorare la safety ma peggiorare la liveness:

- lock unico su tutto il dizionario;
- sezioni critiche troppo lunghe;
- I/O di rete fatto sotto lock;
- sleep o operazioni lente protette inutilmente.

Regola pratica della tappa: proteggere tutto cio' che tocca stato condiviso, ma non trattenere il lock piu' del necessario.

Una richiesta INCR puo' essere letta come automa:

- Idle -> ReadCurrent -> ComputeNext -> WriteBack -> Reply

Nella versione safe:

- Idle -> AcquireLock -> ReadCurrent -> ComputeNext -> WriteBack -> ReleaseLock -> Reply

Domanda chiave: quale sottosequenza deve essere indivisibile per difendere il contratto?

Dal lato client la sintassi resta quasi identica. Ma il significato di una risposta positiva dipende ora anche da:

- quali interleaving sono ammessi;
- quali sezioni sono serializzate;
- quali stati intermedi possono essere osservati o esclusi.

Il contratto del servizio non e' piu' solo protocollo di rete.

Sequenza suggerita:

- ➊ avviare la variante unsafe;
- ➋ usare `stress_incr.py`;
- ➌ osservare un valore finale minore dell'atteso;
- ➍ ripetere sulla variante safe;
- ➎ confrontare correttezza e costo.

Durante il laboratorio vale la pena chiedersi:

- quale stato e' locale al thread e quale e' condiviso;
- quali interleaving violano safety;
- quali interleaving restano leciti ma peggiorano solo prestazioni;
- se KEYS e GET debbano stare sotto lo stesso lock delle scritture.

In un software concorrente l'API non basta a spiegare il comportamento del sistema. Contano anche:

- gli interleaving ammessi;
- le porzioni di codice rese atomiche;
- le proprietà di safety e liveness difese dall'implementazione.

Se non sapete dire quale transizione state proteggendo, il lock da solo non vi salva.